

Problem A. Array Similarity

Choose a sufficiently large prime P , and choose r uniformly at random from $\{0, 1, \dots, P-1\}$. For an interval $[L, R]$, define

$$h(L, R) = \sum_{\substack{i \in [L, R] \\ A_i = \max(A_L, A_{L+1}, \dots, A_i)}} r^i \pmod{P}.$$

For a query comparing $[L_1, R_1]$ and $[L_2, R_2]$, compare

$$h(L_1, R_1)r^{-L_1} \quad \text{and} \quad h(L_2, R_2)r^{-L_2}.$$

The two sides are polynomials in r of degree at most N . They are identical if and only if the two arrays are similar, so by the Schwartz-Zippel lemma, the error probability per query is at most N/P .

It remains to compute $h(L, R)$ efficiently. Set $A_0 = \infty$. For each i , let

$$l_i = \min\{j \mid A_j > A_i\} + 1, \quad r_i = i.$$

Then A_i contributes to $h(L, R)$ exactly for $L \in [l_i, r_i]$. Sweep L from N down to 1 and maintain the active contributions r^i in a Binary Indexed Tree. At the moment when a query's left endpoint is processed, its interval sum gives $h(L, R)$.

The time complexity is $O((N + Q) \log N)$.

Problem B. Bracket Character Frequency

Rephrase the problem as follows. We need N sequences $X_1, \dots, X_N$, each of length $2K$, whose entries are 1 or -1 , such that every prefix sum is non-negative, every total sum is zero, and for each position j , exactly A_j of the sequences have value 1.

Build the sequences position by position. At position k , assign the A_k values equal to 1 to the sequences whose current prefix sums are as small as possible. By induction, after each step the difference between the maximum and minimum prefix sums is at most 2, and this greedy assignment maximizes the number of non-negative prefix sums. If the greedy construction makes some prefix sum negative, no construction can work.

Now assume the greedy construction satisfies all prefix constraints. The total-sum condition requires

$$\sum_{j=1}^{2K} (2A_j - N) = 0.$$

If this holds, the bounded spread of prefix sums implies that all final sums are zero. Hence this condition is also sufficient.

The test takes $O(K)$ time.

Problem C. Card Deck

Think about one card that eventually enters the bag. If immediately before an operation this card is among the top K cards, we may put it into the bag at that operation; delaying it never increases the set of possible final bags. Thus each operation can be regarded as:

- look at the top K cards and mark one unmarked card;
- put all still-unmarked cards among the top K into the bag.

Let $S_0 = \{1, 2, \dots, K\}$. The choices over M operations correspond one-to-one with nested sets

$$S_0 \supseteq S_1 \supseteq \dots \supseteq S_M,$$

where S_i is the set of initially top- K cards that remain unmarked after operation i . For every such chain, the contribution to the answer is $\sum_i |S_i|$.

For a fixed element $x \in S_0$, its contribution is the largest j such that $x \in S_j$. The number of chains in which this contribution equals j is $(M+1)^{K-1}$, independently of j , because the other $K-1$ elements choose their disappearance times freely. Therefore one element contributes

$$\sum_{j=0}^M j(M+1)^{K-1} = \frac{M(M+1)}{2}(M+1)^{K-1}.$$

Multiplying by K , the answer is

$$\frac{KM(M+1)^K}{2}.$$

Problem D. Digits of Prefix Product

One construction uses

$$a_i = 10^{2^{i-1}} - 1 \quad (i \geq 1).$$

For every k , the product

$$b_k = \prod_{i=1}^k a_i$$

can be written as a signed sum whose coefficients are determined by the parity of popcount, i.e. by the Thue-Morse sequence. Since this sequence never has three equal consecutive values, the decimal representation of b_k never has two adjacent equal digits.

To satisfy the additional no-zero digit condition, group the first few a_i and multiply that group by 2 (or by another small factor such as 3 or 4). The shape of the decimal representation above guarantees that the zero digits disappear while the other conditions are preserved. One explicit construction has total length 65536 digits.

There is also a shorter tester construction. Let

$$a_1 = 1818 \dots 1818 \quad (100 \text{ digits}),$$

and for $i \geq 2$ let a_i be a string of 9s whose length doubles each time: 100, 200, 400, $\dots$, 25600. The total length is 51200. The induction proof checks the subtraction

$$b_{i+1} = b_i \cdot 10^{\text{digits}(b_i)} - b_i.$$

Borrowing occurs only in the middle, and the boundary patterns 18 and 82 ensure that no adjacent equal digits are created.

Problem E. Edge Coloring Problem

For an even number of vertices, say $2N$, at least $2N - 3$ colors are necessary, and this bound is always achievable by decomposing the graph into $2N - 3$ perfect matchings.

Look at the complement graph. Its connected components are cycles. If all cycles are even, the vertices can be split into two parts of size N so that the original graph consists of:

- the clique inside the left part;
- a regular bipartite graph of degree $N - 2$ between the two parts;
- the clique inside the right part.

If N is even, each clique decomposes into $N - 1$ perfect matchings and the regular bipartite graph decomposes into $N - 2$ perfect matchings by Hall's theorem. If N is odd, use the standard near-perfect matching decomposition of odd cliques and combine one bipartite perfect matching with the isolated vertices.

If odd cycles exist in the complement, their number is even. Pair them and split each pair in a W-shaped way so that the same argument applies after a local edge replacement; the decomposition can then be converted back to the original graph.

For an odd number of vertices, at least $N - 2$ colors are necessary. Add one new vertex and a few auxiliary edges to obtain an even-vertex regular instance, color it by the method above, and then remove the auxiliary vertex and edges.

The bottleneck is decomposing a regular bipartite graph into perfect matchings; using bipartite matching $O(N)$ times gives an $O(N^{3.5})$ construction.

Problem F. Fourier Coefficients

Using

$$\cos x = \frac{e^{ix} + e^{-ix}}{2},$$

write

$$f(x) = \sum_{k=-(N-1)}^{N-1} B_k e^{ikx}, \quad B_k = \begin{cases} A_{|k|}/2, & k \neq 0, \\ A_0, & k = 0. \end{cases}$$

Thus the goal is to recover all B_k .

Let

$$g(y) = \sum_{k=-(N-1)}^{N-1} B_k y^k,$$

so that $f(x) = g(e^{ix})$. Choose a complex number z on the unit circle whose real and imaginary parts are rational and well-defined modulo 998244353, and such that $z, z^2, \dots, z^{N-1}$ are all non-one in the corresponding finite field extension.

Query

$$g(1), g(z), g(z^2), \dots, g(z^{N-1}).$$

Since $B_{-k} = B_k$, we also know $g(z^{-t}) = g(z^t)$. Therefore values at the $2N - 1$ geometric points

$$z^{-N+1}, \dots, z^{-1}, 1, z, \dots, z^{N-1}$$

are known. Polynomial interpolation on a geometric progression recovers all B_k in $O(N \log N)$ time.

Problem G. Guarding Plan

First remove every initially unnecessary guard. If guard x is watched by another guard y , then y never disappears before x can become necessary, and any construction using x can be dominated by a construction using y instead.

After this reduction, it is enough to place new guards on the upper convex hull, using adjacent guards on that hull. The problem decomposes between adjacent hull guards. After translation, one subproblem has guards $A = (0, a)$ and $B = (b, 0)$, and all other relevant guards lie strictly inside the right triangle AOB .

An interior guard x can be deleted by placing a guard at any point of a closed interval on segment AB ; call this its deletion interval. Since dominated guards were removed, no deletion interval contains another. Hence, after sorting by left endpoint, the right endpoints are also sorted.

The subproblem becomes one-dimensional: given sorted intervals $[a_i, b_i]$, delete all intervals with operations of two types: choose a point and delete all intervals containing it, or delete one interval individually. Minimize first the total number of operations, and then the number of point-choosing operations.

When considering the first remaining interval $[a_1, b_1]$, if it is deleted by a point operation, choosing b_1 is always at least as good as choosing any earlier point in the interval. Thus a DP over prefixes suffices:

$$dp[i] = \text{best pair after deleting intervals } 1, \dots, i.$$

For a point operation at b_i , binary search the farthest interval also containing b_i . This gives an $O(N \log N)$ algorithm. The required comparisons can be implemented exactly without floating point.

Problem H. Hidden Sequence Rotation

Query all length-one substrings $(s_i, l_i) = (i, 1)$ to find the positions of the minimum value. If every position is minimum, then every rotation is optimal.

Otherwise let

$$T_{\text{cand}} = \{x_1, x_2, \dots, x_m\}$$

be the positions where A_i is minimum but A_{i-1} is not. Every lexicographically minimum rotation starts at one of these positions. Split the array into blocks

$$B_j = A(x_j, x_{j+1} - x_j).$$

Because each block begins with the minimum value and ends just before the next minimum run begins, comparing rotations $A(x_i, N)$ and $A(x_j, N)$ is equivalent to comparing the cyclic sequences of blocks

$$(B_i, B_{i+1}, \dots) \quad \text{and} \quad (B_j, B_{j+1}, \dots)$$

lexicographically.

Therefore we can compress each block into one symbol and repeat the same procedure on the shorter sequence. Since no two candidate positions are adjacent in the original sequence, the length is at most halved each round. After at most $\lceil \log_2 N \rceil$ rounds, the answer is determined. The total query length constraint is preserved because each round queries disjoint blocks.

Problem I. Insert AB or BA

For a string U over A and B, define the cumulative-sum sequence $A(U)$ by replacing A with +1, B with -1, and taking prefix sums.

Assume $|S| \leq |T|$. The string S can be transformed into T if and only if there exists a strictly increasing map

$$f : \{1, \dots, |S|\} \rightarrow \{1, \dots, |T|\}$$

such that

$$S_i = T_{f(i)}, \quad A(S)_i = A(T)_{f(i)}.$$

The map records where each original character of S appears in T . Insertions of AB or BA preserve the cumulative sum outside the inserted segment, giving the second condition.

If such a map is fixed, the number of AB insertions needed between S_i and S_{i+1} is

$$\max_{f(i) \leq j < f(i+1)} A(T)_j - A(S)_i.$$

The same formula applies at the beginning and end. Thus minimizing cost becomes a DP over possible values of $f(i)$.

The naive DP is $O(|T|^3)$. The expression above can be maintained with a stack of pairs consisting of current minimum cost and current maximum prefix sum, reducing the complexity to $O(|T|^2)$.

Problem J. Journey through the Fractal

Call the central triangle created at each level downward, and the other triangles upward. Every triangle adjacent to an upward triangle is downward, and every triangle adjacent to a downward triangle is upward, so Alice alternates between upward and downward triangles.

All upward triangles have level 1. A downward triangle of level l appears 3^{L-l} times. Hence the goal is to maximize the total level of visited downward triangles.

A key bound is: a triangle of level $L - t$ can be visited at most 2^t times. This is proved by induction. For $t = 0$ there is only the single level- L triangle. For the step, visiting level $L - t$ triangles can be grouped by the containing level $L - t + 1$ fractal, and two such fractals cannot be visited consecutively without visiting a triangle of level at least $L - t + 1$ in between.

The bound is tight. Choose downward triangles greedily by level under these caps, and place the chosen triangles along one outer side of the fractal. In that order, every consecutive pair has an adjacent upward triangle, so the walk exists.

Since the cap doubles by level, only $O(\log K)$ levels need to be simulated. Also note that if L is small and K is large, fewer than K moves may be possible.

Problem K. K-rep Array

Use zero-based indexing. A sequence V is K -rep exactly when

$$V_i = V_{i+K} \quad (0 \leq i < N - K),$$

so V_i depends only on $i \bmod K$. Therefore A cannot be completed to a K -rep array if there exist known entries $A_i \neq A_j$ with $i - j$ a multiple of K .

First compute the set S of all distances $|i - j|$ for which A_i and A_j are both known and unequal. This can be done by wildcard convolution. Replace every -1 by 0 and define

$$f(x, y) = xy(x - y)^2 = x^3y - 2x^2y^2 + xy^3.$$

Let A' be the reversed array, and convolve the arrays needed for the three monomials. The coefficient is nonzero exactly when there is a pair of known unequal values at the corresponding distance.

Once S is known, for each K check whether any element of S is a multiple of K . This can be done by enumerating multiples for all K in harmonic-series time, or by a Möbius-transform view.

The total complexity is $O(N \log N)$. Because the convolution values can exceed 64-bit integers, compute them modulo several primes whose product exceeds the theoretical upper bound.

Problem L. LIS Triangle

When $K = 1$, no valid permutation exists: among any three consecutive positions, a side of length 1 appears, so a non-degenerate triangle cannot always be formed.

For $K = 2$, we can always construct a valid permutation. For $2 \leq k \leq N$, define

$$Q = (2, 3, \dots, k, N + 1, N, \dots, k + 1).$$

The LIS length of Q is k . If $k = 2$ or $k - 1 + k > N + 1$, then Q also satisfies the triangle condition. Hence this covers $L > N/2 + 1$.

For $L < N/2$, use the reversed construction

$$Q' = (k + 1, \dots, N, N + 1, k, \dots, 3, 2),$$

whose LIS length is $N + 1 - k$.

The remaining middle cases $N/2 \leq L \leq N/2 + 1$ are handled by slight variants

$$R = (3, 4, \dots, k, N + 1, N, \dots, k + 1, 2)$$

and its reverse-like counterpart. For example, these cover the central cases that Q and Q' miss.

For $K \geq 3$, take the $K = 2$ construction and add $K - 2$ to every element.

Problem M. Minimum Distance Tree

Let T' be a minimum spanning tree of G , obtained by Kruskal's algorithm. For every edge (u_i, v_i, w_i) of T' , the cut property implies that any walk in G connecting u_i and v_i must contain an edge of weight at least w_i .

Assume a valid tree T exists. If T did not contain the edge connecting u_i and v_i , then the path between them in T would use at least two positive-weight edges, each of weight less than w_i . Since each edge of T represents a shortest distance in G , those edges would give a walk in G from u_i to v_i using only weights below w_i , contradicting the cut property. Hence every MST edge must appear in T , and therefore $T = T'$.

It remains to verify whether T' satisfies the condition. For every non-tree edge (u_j, v_j, w_j) , compute the distance between u_j and v_j in T' . If it is larger than w_j , the condition fails. Otherwise that edge can be removed from consideration without changing shortest distances. If all non-tree edges pass this test, T' is valid.

Build the MST in $O(M\alpha(N) + M \log M)$ time. Path-distance queries in T' are answered by LCA preprocessing in $O(M \log N)$ total time.

Problem N. Nice Bouquets

Assign values 0, 1, 2 to the three colors. A multiset of flowers collected on one day can be partitioned into valid bouquets if and only if both the number of flowers and the sum of their color values are divisible by 3.

For each tree, form a vector over $\mathbb{F}_3$ of dimension $K + 1$: the first K coordinates are the colors over the next K days, and the last coordinate is 1. For all days to be valid, the sum of the selected vectors must be zero. Cutting a tree means assigning coefficient 0, leaving it means coefficient 1, and applying growth agent means coefficient 2.

Subtract 1 from all coefficients. Let

$$u = - \sum_i v_i.$$

Now the task is to find the smallest index i such that u lies in the span of $v_1, \dots, v_i$ over $\mathbb{F}_3$. If $u = 0$, the answer is 0.

Scan trees in increasing order and maintain a linear basis over $\mathbb{F}_3$. After inserting each vector, test whether the basis spans u . Gaussian elimination costs $O(K)$ per basis vector, and the basis size is at most $\min(N, K)$.

The time complexity is $O(NK \min(N, K))$, which is fast enough under the constraints.

Problem O. One Different Inequality

Call two adjacent terms a good pair if their difference is 1, and a bad pair otherwise. Split the inequality string into maximal alternating parts. In a part of length l , at least $\lfloor l/2 \rfloor$ bad pairs are necessary, and this lower bound is achievable. For odd l , the choice is unique; for even l , there are multiple choices.

If the bad pairs are fixed, they act as separators, and each remaining block consists of consecutive values. The remaining problem is to count possible relative orders of these blocks, which is the classical count of

permutations matching a pattern of inequalities. This is solvable by insertion DP.

For the full solution, the problem becomes: sum the number of good permutations over all inequality strings consistent with several uncertain intervals of the forms $\ll \dots \gg$ or $\gg \dots \ll$. Without uncertain intervals, inclusion-exclusion plus divide-and-conquer FFT gives $O(n \log^2 n)$.

Apply inclusion-exclusion to the $>$ positions and run the same DP on strings over $<$ and unknown symbols. For an uncertain interval of type $\ll \dots \gg$, entering the interval multiplies the transition by a factor depending only on parity from the left endpoint; for type $\gg \dots \ll$, the analogous multiplier is applied before leaving. Transitions jumping over an odd-length uncertain interval cancel to zero, while even-length jumps are unchanged.

All of these effects can be implemented by multiplying by 1, 0, or -1 before or after the convolutions in the divide-and-conquer FFT. The complexity is $O(n \log^2 n)$.

Problem P. Perfect Suika Game on a Tree

Write 2^{A_v} on vertex v . Contracting an edge whose endpoints have the same value x creates a vertex with value $2x$. Thus if the whole tree can be contracted to one vertex, the final value is

$$\sum_v 2^{A_v},$$

which must be a power of two. If not, every query answer is No.

For the decision itself, process values in increasing powers of two. At each level 2^k , after all smaller levels have been contracted, the vertices currently labeled 2^k must induce a forest with a perfect matching. In a forest, such a perfect matching, if it exists, is unique, so a direct simulation from the smallest level upward gives a linear-time solution for the static version.

For the full query version, use the following rooted-tree fact. Let black vertices be the current vertices of one level. Let $f(T)$ be the number of black vertices, and let $g(T)$ be the number of vertices whose rooted subtree contains an odd number of black vertices. Then

$$\left\lfloor \frac{f(T) + 1}{2} \right\rfloor \leq g(T),$$

and equality holds exactly when the black-induced subgraph has a perfect matching.

For every level k , it is enough to know the number of vertices of value 2^k after lower contractions, and the corresponding g value. The first depends only on level counts. For the second, a vertex contributes to level k according to the lowest set bit of the sum of 2^{A_v} over its subtree; contraction preserves this sum.

Maintain sets of active bit runs for these subtree sums. For swap queries, only one subtree sum changes for the endpoint on the leaf side of the swapped edge. Representing consecutive set bits as intervals allows both carry and borrow operations in logarithmic time.

The total complexity is $O((N + Q) \log^2(N + Q))$. An implementation variant uses two priority queues to support adding or subtracting powers of two and asking for the lowest set bit, amortized logarithmically.

Problem Q. Quadratic Pieces

A key property is that every contiguous subsequence of a quadratic sequence is also quadratic. Therefore the following greedy strategy is correct:

- find the longest quadratic contiguous prefix of the current sequence;
- cut it off and repeat.

The longest prefix can be found by scanning. One way is to determine the quadratic polynomial from the first three elements and then extend while the values match it.

A simpler implementation uses second differences. Define

$$B_i = A_i - 2A_{i+1} + A_{i+2}.$$

Then a segment $(A_L, A_{L+1}, \dots, A_R)$ is quadratic if and only if

$$B_L = B_{L+1} = \dots = B_{R-2}.$$

Thus each greedy piece is just a maximal run of equal values in the second-difference array, with the obvious handling of segments of length at most two.